



Department of Physics, IIT Indore

Weekly Report

AUTHOR

Prince Kumar (230051013)

SUPERVISOR

Dr. Shashank Gupta

2026-05-17

Abstract

This weekly report documents the progress made in Week 2 of the quantum key distribution (QKD) network simulation project. The primary focus during this week was completing the Discrete Event Simulation (DES) tutorial from the Sequence framework documentation. The tutorial provided a comprehensive foundation in event-driven simulation principles through practical implementation of a Store class. The report covers the key concepts learned, including Timeline management, Event scheduling, Process handling, and logging mechanisms within the Sequence framework. These foundational concepts are essential for building the subsequent components of the QKD network simulation.

Contents

Table of Contents ii

1 Introduction 1

2 Activities and Findings 1

 2.1 Discrete Event Simulation Overview 1

 2.1.1 Core Components 1

 2.2 Practical Implementation: Store Class 2

 2.2.1 Basic Store Implementation 2

 2.2.2 Event Scheduling 3

 2.2.3 Automated Event Scheduling 4

 2.2.4 Timeline Termination 5

 2.2.5 Logging and Debugging 5

3 Key Findings and Concepts 7

4 Implementation Artifacts 8

5 Relevance to QKD Simulation 8

6 Next Steps and Future Work 9

7 Conclusion 9

1 Introduction

This report documents the second week of the quantum key distribution (QKD) network simulation project. As outlined in the previous week's recommendations, the focus of this week was to gain a comprehensive understanding of Discrete Event Simulation (DES) systems through the official Sequence framework tutorial. DES is a fundamental paradigm for simulating systems where events occur at discrete points in time, and the state of the system changes only in response to these events. This understanding is critical for implementing the QKD network simulation, which will rely heavily on event-driven architecture.

2 Activities and Findings

2.1 Discrete Event Simulation Overview

The Discrete Event Simulation tutorial from the Sequence framework provided a practical introduction to event-driven programming. The tutorial demonstrated how to build a simple but realistic simulation system using the core components of the Sequence kernel.

2.1.1 Core Components

The DES implementation in Sequence relies on several key components:

- **Timeline:** The Timeline class manages the simulation time and maintains a queue of scheduled events. It acts as the central coordinator for the entire simulation.
- **Event:** An Event represents an occurrence at a specific point in simulation time. Each event contains a timestamp and a process to be executed.

- **Process:** A Process encapsulates a function call with its associated arguments. Processes are executed when their corresponding events are triggered.

2.2 Practical Implementation: Store Class

The tutorial provided a practical example through the implementation of a Store class that simulates a retail store opening and closing operations. The implementation evolved through several stages of increasing complexity and functionality.

2.2.1 Basic Store Implementation

The initial implementation established the foundation:

```
from sequence.kernel.timeline import Timeline
```

```
class Store(object):  
    def __init__(self, tl: Timeline):  
        self.opening = False  
        self.timeline = tl  
  
    def open(self) -> None:  
        self.opening = True  
  
    def close(self) -> None:  
        self.opening = False
```

The basic Store class maintains an `opening` state and provides methods to transition between open and closed states. The Timeline object provides access to simulation time and scheduling capa-

bilities.

2.2.2 Event Scheduling

The tutorial demonstrated how to schedule events on the Timeline:

```
from sequence.kernel.event import Event
from sequence.kernel.process import Process

tl = Timeline()
store = Store(tl)

# Create and schedule an opening event at time 7
open_proc = Process(store, 'open', [])
open_event = Event(7, open_proc)
tl.schedule(open_event)

# Run the simulation
tl.run()

print(tl.now(), store.opening) # Output: 7 True
```

This demonstrates the basic workflow: create a Timeline, instantiate objects, create processes that reference methods on those objects, wrap processes in events with specific timestamps, schedule events, and run the simulation.

2.2.3 Automated Event Scheduling

The tutorial progressed to a more sophisticated implementation where events schedule subsequent events automatically:

```
class Store:

    def __init__(self, tl: Timeline):
        self.opening = False
        self.timeline = tl

    def open(self) -> None:
        self.opening = True
        # Schedule closing 12 time units from now
        process = Process(self, 'close', [])
        event = Event(self.timeline.now() + 12, process)
        self.timeline.schedule(event)

    def close(self) -> None:
        self.opening = False
        # Schedule opening 12 time units from now
        process = Process(self, 'open', [])
        event = Event(self.timeline.now() + 12, process)
        self.timeline.schedule(event)
```

This version demonstrates event-driven cascading, where each event triggers the scheduling of the next event, creating a continuous cycle of state transitions.

2.2.4 Timeline Termination

The tutorial also covered simulation control mechanisms using Timeline termination parameters:

```
tl = Timeline(60)  # Simulation ends at time 60
tl.show_progress = False  # Disable progress bar
store = Store(tl)
process = Process(store, 'open', [])
event = Event(7, process)
tl.schedule(event)
tl.run()
```

2.2.5 Logging and Debugging

The final version integrated the Sequence logging framework for better visibility into simulation execution:

```
import sequence.utils.log as log

class Store:
    def __init__(self, tl: Timeline):
        self.opening = False
        self.timeline = tl

    def open(self) -> None:
        if self.timeline.now() >= 60:
            self.timeline.stop()
```



```
log.logger.info('Store being opened.')
if self.opening == True:
    log.logger.warning('Store was already open.')

self.opening = True
process = Process(self, 'close', [])
event = Event(self.timeline.now() + 12, process)
self.timeline.schedule(event)

def close(self) -> None:
    if self.timeline.now() >= 60:
        self.timeline.stop()

log.logger.info('Store being closed.')
if self.opening == False:
    log.logger.warning('Store was already closed.')

self.opening = False
process = Process(self, 'open', [])
event = Event(self.timeline.now() + 12, process)
self.timeline.schedule(event)
```

3 Key Findings and Concepts

The Discrete Event Simulation tutorial provided several important insights:

- **Timeline Management:** The Timeline class provides a robust mechanism for managing simulation time and event scheduling. It automatically processes events in chronological order, eliminating the need for manual time management.
- **Event Encapsulation:** By encapsulating state changes in discrete events, complex systems can be modeled as sequences of state transitions. This abstraction simplifies reasoning about system behavior.
- **Cascading Events:** Events can schedule subsequent events, enabling the creation of complex behavioral patterns without explicit control flow. This pattern is fundamental to discrete event simulation.
- **Simulation Time vs. Wall-Clock Time:** Simulation time (as tracked by the Timeline) is independent of wall-clock time, allowing simulations to run at arbitrary speeds.
- **Logging and Observability:** The Sequence framework provides built-in logging capabilities that are essential for understanding and debugging simulations. Logs are timestamped with simulation time, providing a clear record of events in chronological order.
- **State Validation:** The Store example demonstrated the importance of state validation through warnings when state transitions occur unexpectedly (e.g., attempting to open an already-open store).

4 Implementation Artifacts

The DES tutorial implementation has been documented in the following files:

- **des.ipynb:** A Jupyter notebook containing the complete DES tutorial with interactive examples and explanations.
- **des.py:** A standalone Python script implementation of the final Store class, suitable for execution outside the notebook environment. The script demonstrates proper logging configuration and simulation execution.

5 Relevance to QKD Simulation

The Discrete Event Simulation concepts learned this week are directly applicable to the QKD network simulation:

- **Network Events:** In the QKD simulation, network events such as photon transmission, reception, and measurement will be represented as discrete events on the Timeline.
- **Quantum State Management:** The quantum states at different nodes can be managed similarly to the Store's opening/closing state, with events triggering state transitions.
- **Timing Precision:** The Timeline's ability to handle events at precise simulation times is essential for accurately modeling quantum protocols with specific timing requirements.
- **Scalability:** The event-driven architecture demonstrated in the Store example scales naturally to complex systems with multiple nodes and interactions.

6 Next Steps and Future Work

Based on the completion of the DES tutorial, the following activities are planned for the next phase:

1. **Building a Ping-Pong Network System:** Implement a simple network topology where messages are exchanged between nodes (ping-pong pattern) to understand network simulation mechanics within the Sequence framework.
2. **Quantum Component Integration:** Begin studying the Sequence framework's quantum components (quantum channels, quantum memories, etc.) for implementing quantum protocol simulations.
3. **BBM92 Protocol Implementation:** Start development of the Bennet-Brassard-Mermin-92 (BBM92) quantum key distribution protocol using the DES foundations and network architecture learned to date.

7 Conclusion

Week 2 successfully established a comprehensive understanding of Discrete Event Simulation through the completion of the Sequence framework's DES tutorial. The tutorial provided both theoretical concepts and practical implementation experience through the Store class example. The mastery of Timeline management, event scheduling, process handling, and logging mechanisms provides a solid foundation for implementing the complex QKD network simulation. The concepts of cascading events, state management, and event-driven architecture are directly applicable to quantum protocol simulation. With this foundation in place, the project is well-positioned to proceed with implementing the basic network topology and subsequently the quantum key distribution protocols. The DES tuto-

rial has confirmed that the Sequence framework is an appropriate choice for this project, providing all necessary tools for building sophisticated network simulations with quantum components.